SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : P Hemendar Reddy

Reg. No : 11249A295

Class : II B.E. (CSE)

Course Code: BCSF244AI2

Course Name:Data Science and Toolkit Lab

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of the **Data Science Toolkit Lab** with the subject code **BCSF244AI2** was done by

Mr/Ms. P Hemendar Reddy ,

with **Reg. No 11249A295** of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station:Enathur

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

INDEX

SL.NO	ProgramName	Date	Page no	Signature
1	Introduction to python libraries			
a	Numpy			
b	Pandas			
c	Matplotlib			
d	Scikit-learn			
2	Perform data exploration and preprocessing python			
3	Implement Regularized Linear Regression			
4	Implement Naive Bayes Classifier			
5	Implement Regularized Logistic Regression			
6	Build models using ensembling techniques			
7	Build models using decision tree			
8	Support vector machines with different Kernels			
9	Implement K-Nearest Neighbours Classification			
10	K-Means clustering with PCA and elbow method			

Exp 1 : Introduction to python libraries

1.1 Numpy

Aim : to implement array creation and operations using Numpy

11249A295 DSE Record

File Edit View Insert Runtime Tools Help

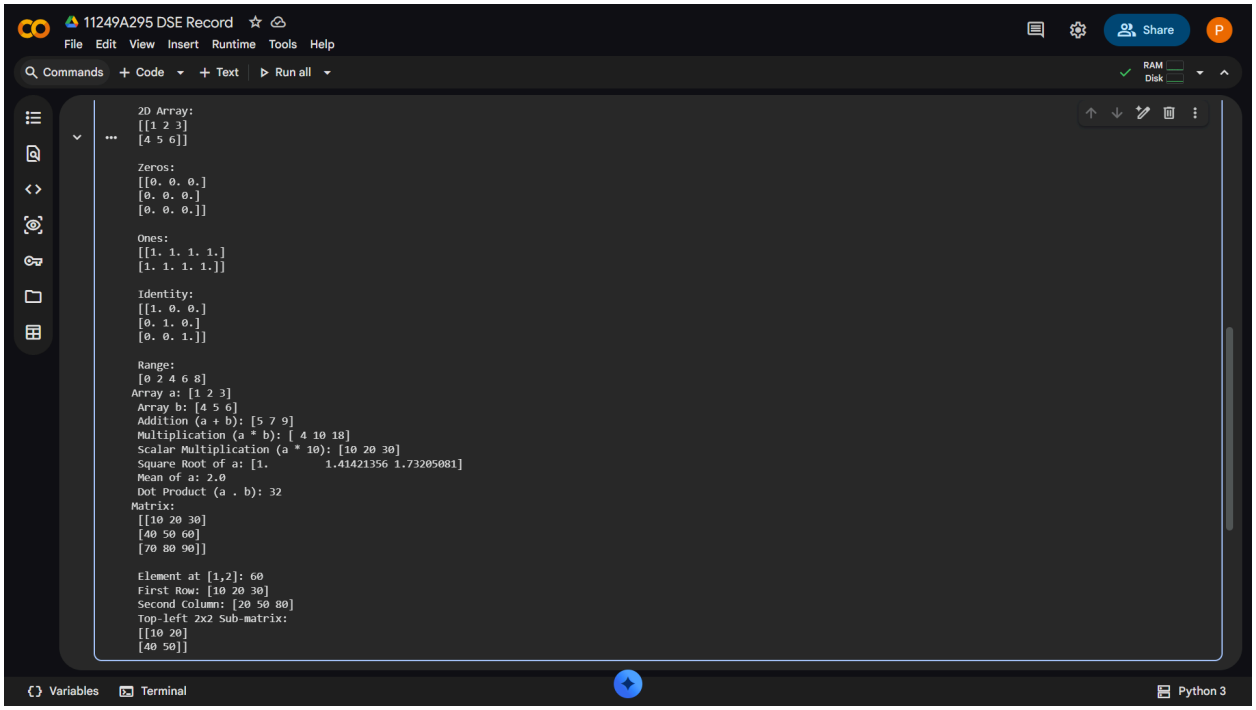
Commands + Code + Text Run all

[1]

```python
import numpy as np
print(
    "1D Array:\n", np.array([1,2,3,4,5]), "\n\n",
    "2D Array:\n", np.array([[1,2,3],[4,5,6]]), "\n\n",
    "Zeros:\n", np.zeros((3,3)), "\n\n",
    "Ones:\n", np.ones((2,4)), "\n\n",
    "Identity:\n", np.eye(3), "\n\n",
    "Range:\n", np.arange(0,10,2)
)
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(
    "Array a:", a, "\n",
    "Array b:", b, "\n",
    "Addition (a + b):", a + b, "\n",
    "Multiplication (a * b):", a * b, "\n",
    "Scalar Multiplication (a * 10):", a * 10, "\n",
    "Square Root of a:", np.sqrt(a), "\n",
    "Mean of a:", np.mean(a), "\n",
    "Dot Product (a . b):", np.dot(a, b)
)
import numpy as np
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
print(
    "Matrix:\n", matrix, "\n\n",
    "Element at [1,2]:", matrix[1, 2], "\n",
    "First Row:", matrix[0, :], "\n",
    "Second Column:", matrix[:, 1], "\n",
    "Top-left 2x2 Sub-matrix:\n", matrix[0:2, 0:2]
)
```

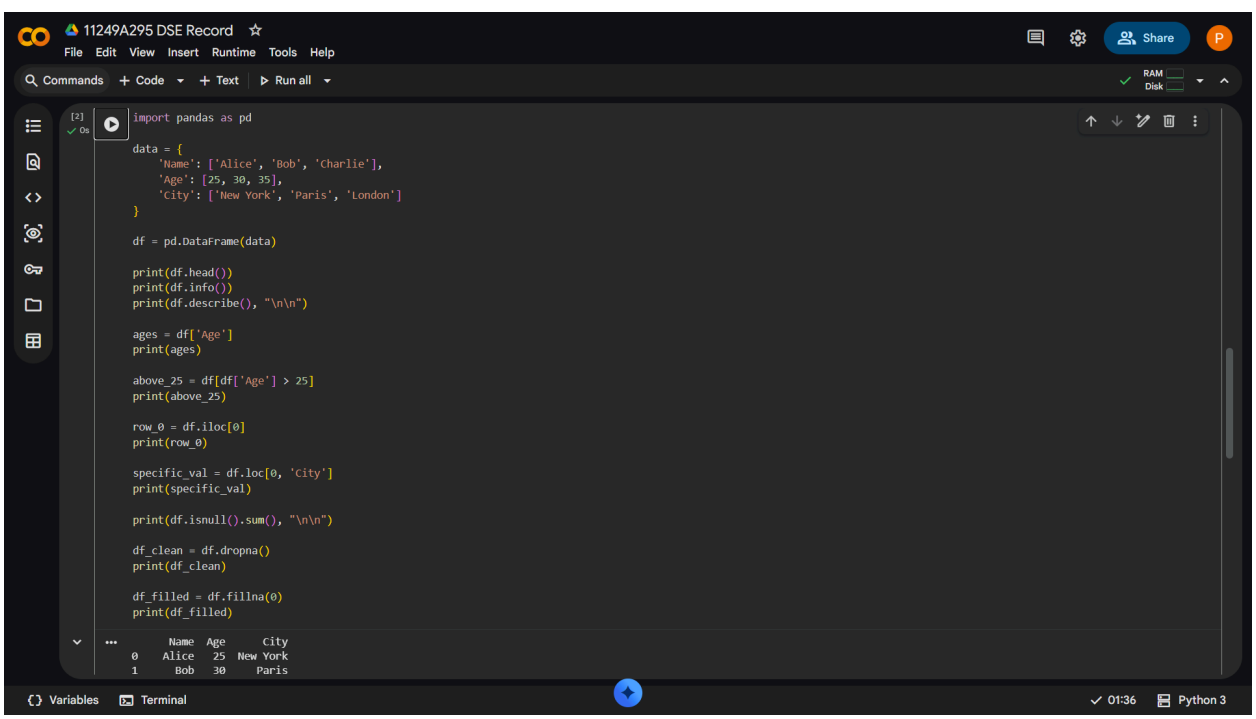
```
1D Array:
```

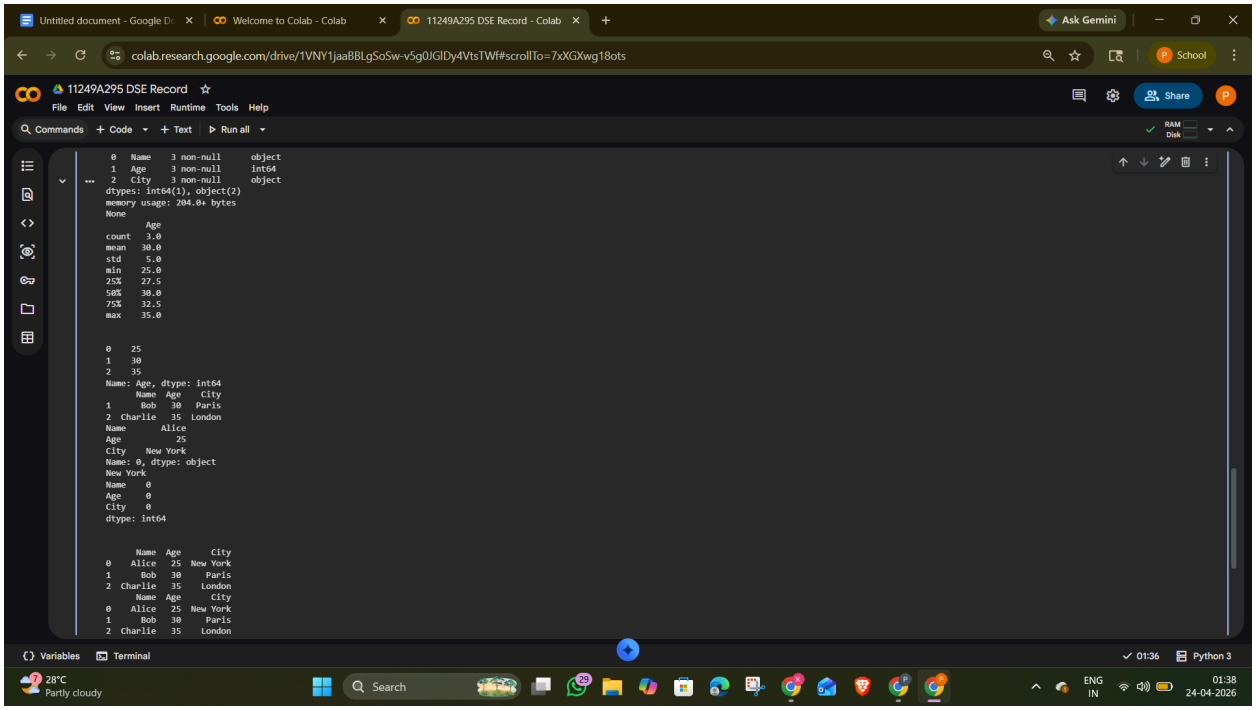
Variables Terminal Python 3



1.2 Pandas

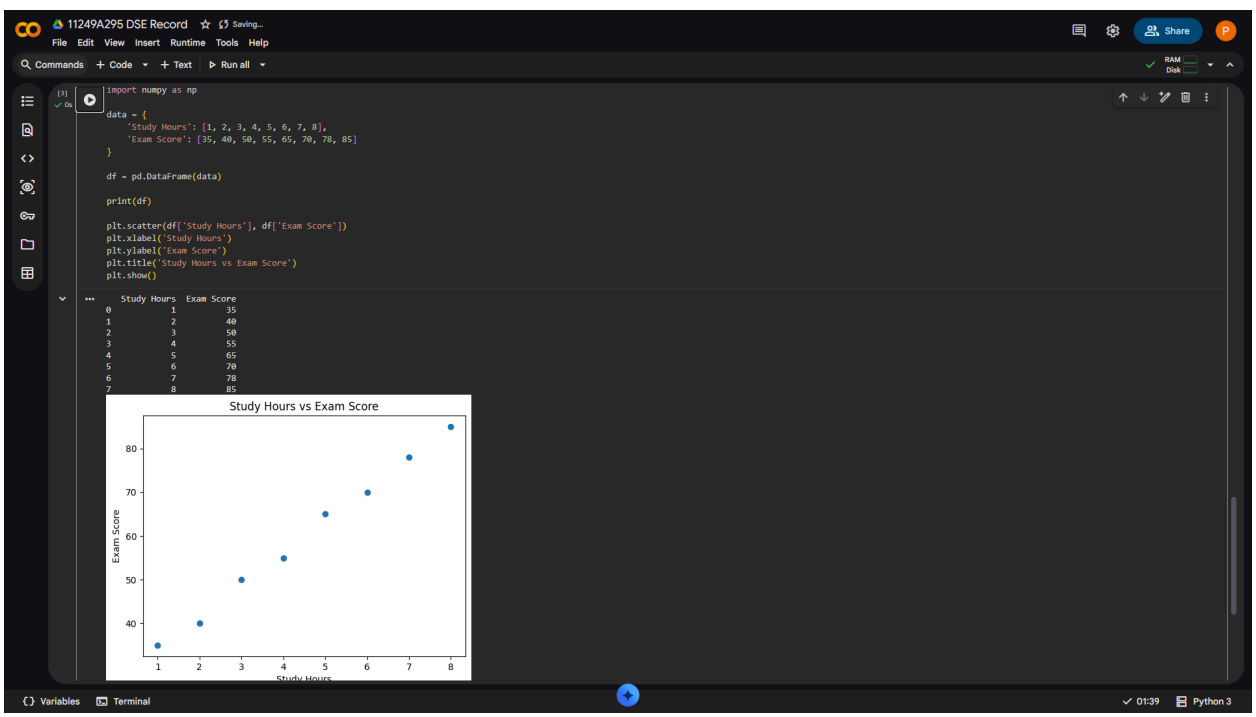
Aim: To implement data creations,filtering,and cleaning using Pandas





1.3 Matplot lib

Aim : To implement plotting operations using Matplolib.



11249A295 DSE Record

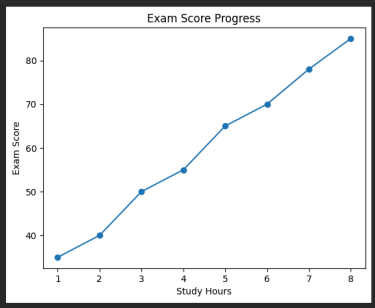
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

To exit full screen, press and hold Esc

[4]

```python
import matplotlib.pyplot as plt
import pandas as pd

# Create dataset
data = {
    'Study Hours': [1, 2, 3, 4, 5, 6, 7, 8],
    'Exam Score': [35, 40, 50, 55, 65, 70, 78, 85]
}

df = pd.DataFrame(data)

# Plot
plt.plot(df['Study Hours'], df['Exam Score'], marker='o')
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Exam Score Progress')
plt.show()
```

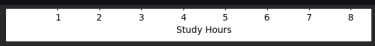


Variables Terminal 01:40 Python 3

11249A295 DSE Record Saving...

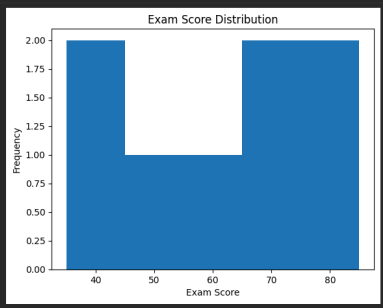
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

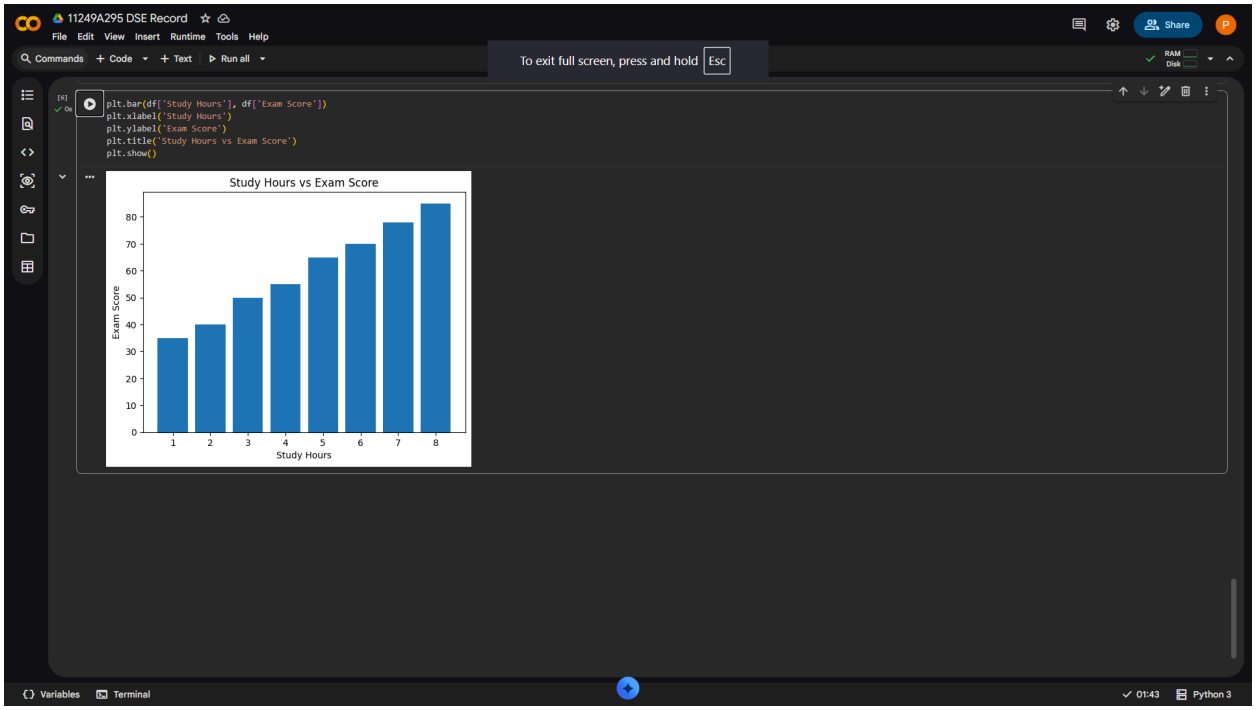


[5]

```python
import matplotlib.pyplot as plt

plt.hist(df['Exam Score'], bins=5)
plt.xlabel('Exam Score')
plt.ylabel('Frequency')
plt.title('Exam Score Distribution')
plt.show()
```

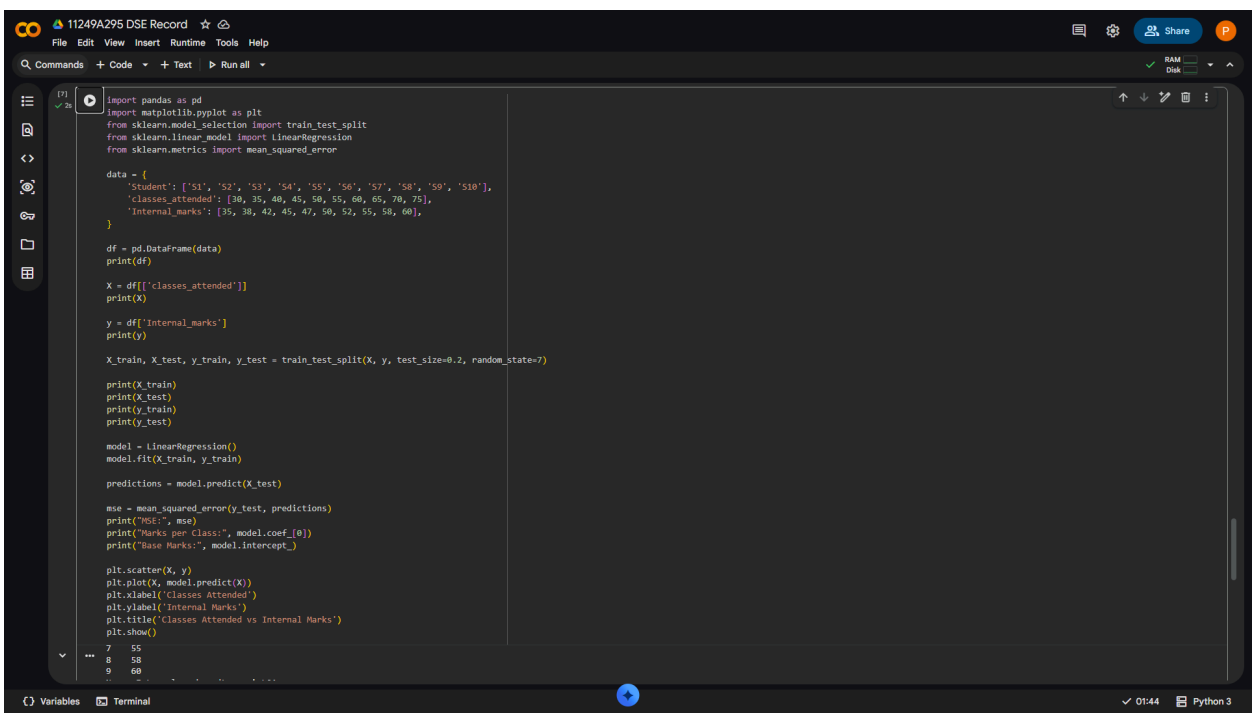


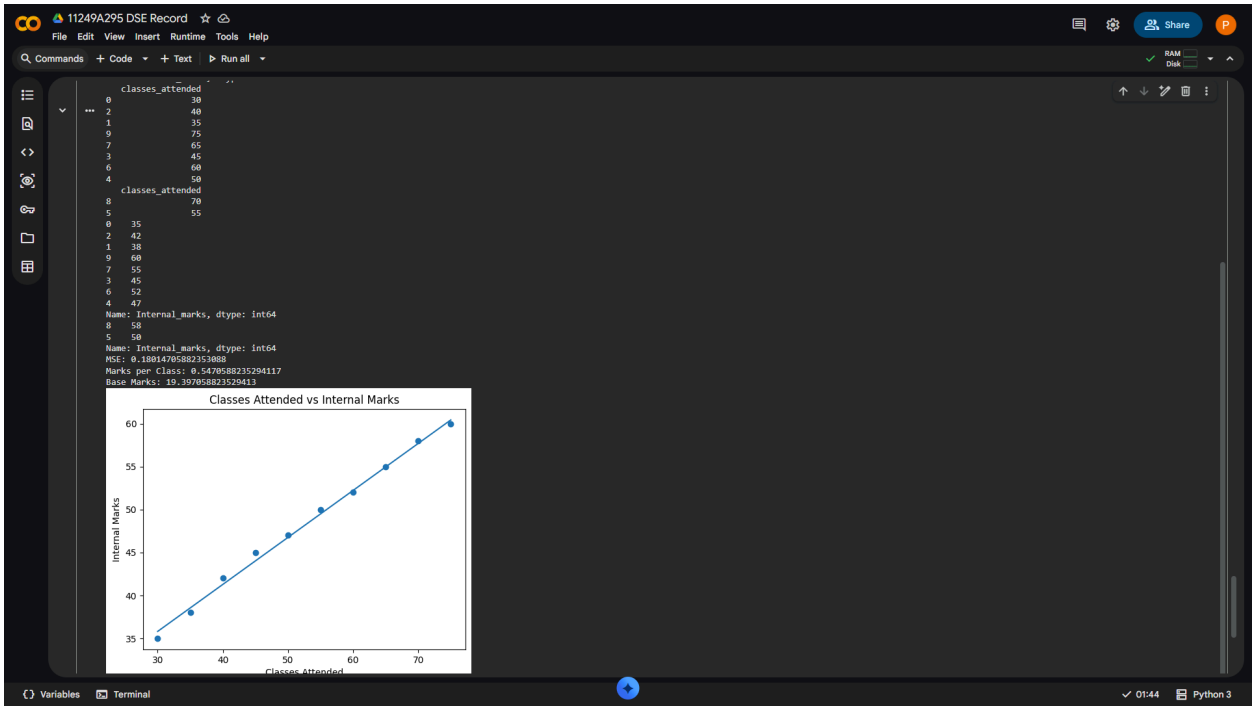
Variables Terminal 01:42 Python 3



1.4 Scikit-Learn

Aim : To implement machine learning algorithms using Scikit-Learn





Result : Machine learning modes were implement successfully.

Exp 2 : Perform Data Exploration and preprocessing in python

Aim : To implement data exploration and preprocessing.

11249A295 DSE Record

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = {
'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
'Age': [22, 25, np.nan, 30, 28, 35],
'Salary': [40000, np.nan, 50000, np.nan, 72000, 58000],
'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
}
df = pd.DataFrame(data)
print("--- ORIGINAL RAW DATA ---")
print(df)
print("\n")

print("--- MISSING VALUES COUNT ---")
print(df.isnull().sum())
print("\n")
df['Age'] = df['Age'].fillna(df['Age'].mode())
df['Salary'] = df['Salary'].fillna(df['Salary'].mode())

print("--- DATA AFTER CLEANING ---")
print(df)
print("\n")
df_encoded = pd.get_dummies(df, columns=['Country'])
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})

print("--- DATA AFTER ENCODING (All Numbers Now) ---")
print(df_encoded)
print("\n")

X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

```
--- ORIGINAL RAW DATA ---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0      NaN       Yes
2   India   NaN  50000.0       Yes
3     USA  30.0      NaN        No
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes
```

Variables Terminal 01:47 Python 3

11249A295 DSE Record

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
X_scaled = scaler.fit_transform(X)
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

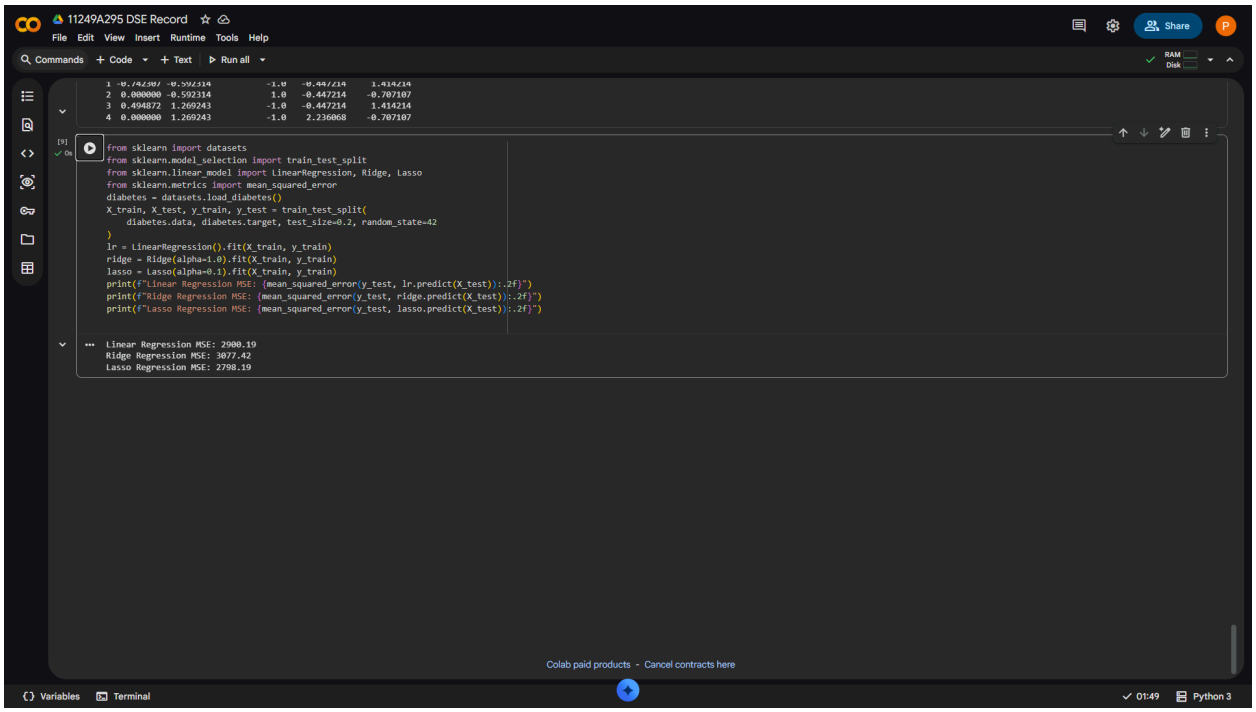
```
--- ORIGINAL RAW DATA ---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0      NaN       Yes
2   India   NaN  50000.0       Yes
3     USA  30.0      NaN        No
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes


--- MISSING VALUES COUNT ---
Country      0
Age          1
Salary       2
Purchased    0
dtype: int64


--- DATA AFTER CLEANING ---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  50000.0       Yes
2   India  28.0  50000.0       Yes
3     USA  30.0  72000.0        No
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0       Yes


--- DATA AFTER ENCODING (All Numbers Now) ---
    Age   Salary  Purchased  Country_India  Country_UK  Country_USA
0  22.0  40000.0          0           True       False        False
1  25.0  50000.0          1          False       False         True
2  28.0  50000.0          1           True       False        False
3  30.0  72000.0          0          False       False         True
4  28.0  72000.0          1          False        True        False
5  35.0  58000.0          1           True       False        False


--- FINAL PROCESSED DATA (Ready for AI) ---
        Age    Salary  Country_India  Country_UK  Country_USA
0 -1.484615 -1.438476           1.0   -0.447214    -0.707107
1 -0.742307 -0.592314          -1.0   -0.447214     1.414214
2  0.000000 -0.592314           1.0   -0.447214    -0.707107
3  0.494872  1.269243          -1.0   -0.447214     1.414214
4  0.000000  1.269243          -1.0    2.236068    -0.707107
```

Variables Terminal 01:47 Python 3

Result : Data preprocessing was performed successfully.

3 : Implement Regularized linear regression.

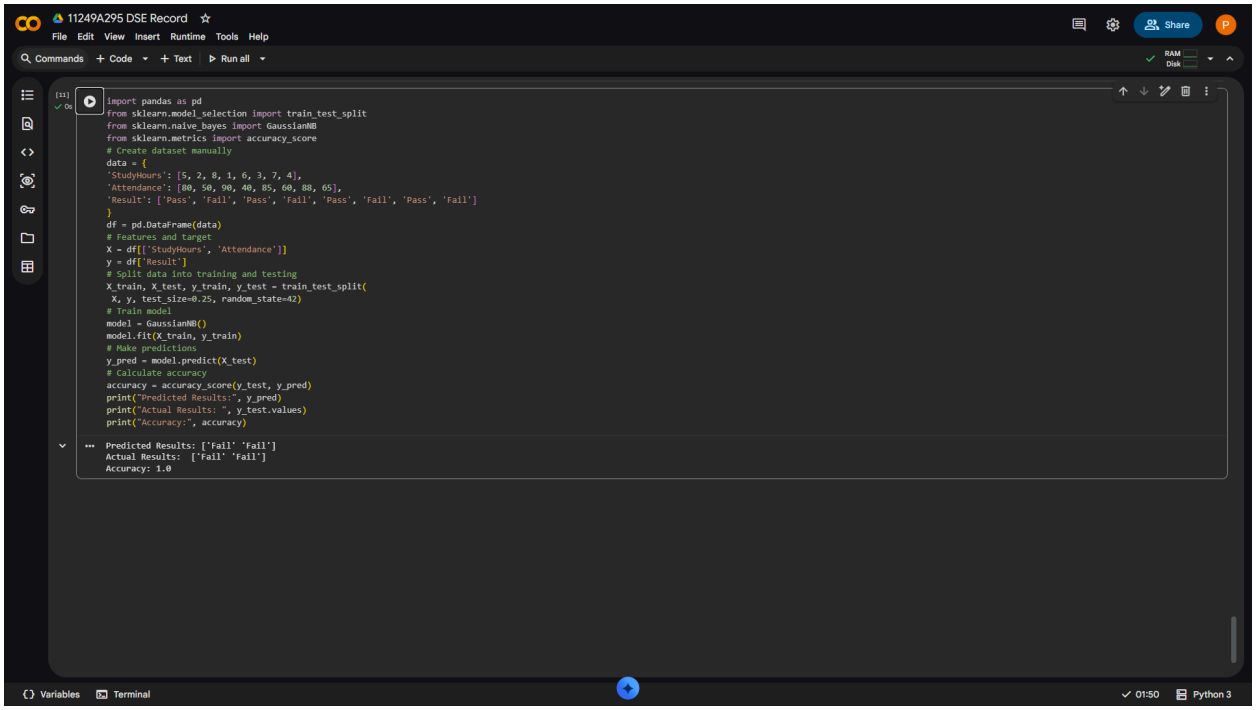
Aim : To implement regularized linear regression.



Result : The regression model was implemented successfully

Exp 4 : Implement Navie Bayes Classifier

Aim : To implement Navie Bayes Classification.



```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
# Create dataset manually
data = {
'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}
df = pd.DataFrame(data)
# Features and target
X = df[['StudyHours', 'Attendance']]
y = df['Result']
# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.25, random_state=42)
# Train model
model = GaussianNB()
model.fit(X_train, y_train)
# Make predictions
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Results:", y_pred)
print("Actual Results: ", y_test.values)
print("Accuracy:", accuracy)
```

```
Predicted Results: ['Fail' 'Fail']
Actual Results:  ['Fail' 'Fail']
Accuracy: 1.0
```

Result : The Classifier was implemented successfully.

Exp 5 : Implement regularized logistic regression

Aim : To implement regularized logistic regression.

11249A295 DSE Record

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 2. Load Dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 3. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Normal Logistic Regression (Very Large C)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)

normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)

# 6. Regularized Logistic Regression
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)

reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

# 7. Print Results
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)

# 8. Compare Coefficients
plt.figure()
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
```

Variables Terminal 01:53 Python 3

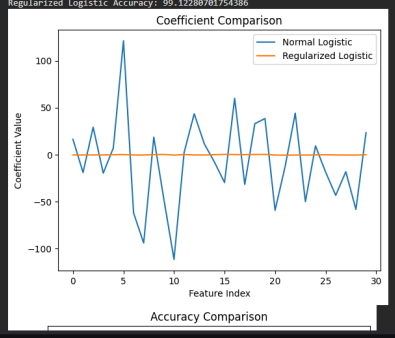
11249A295 DSE Record

File Edit View Insert Runtime Tools Help

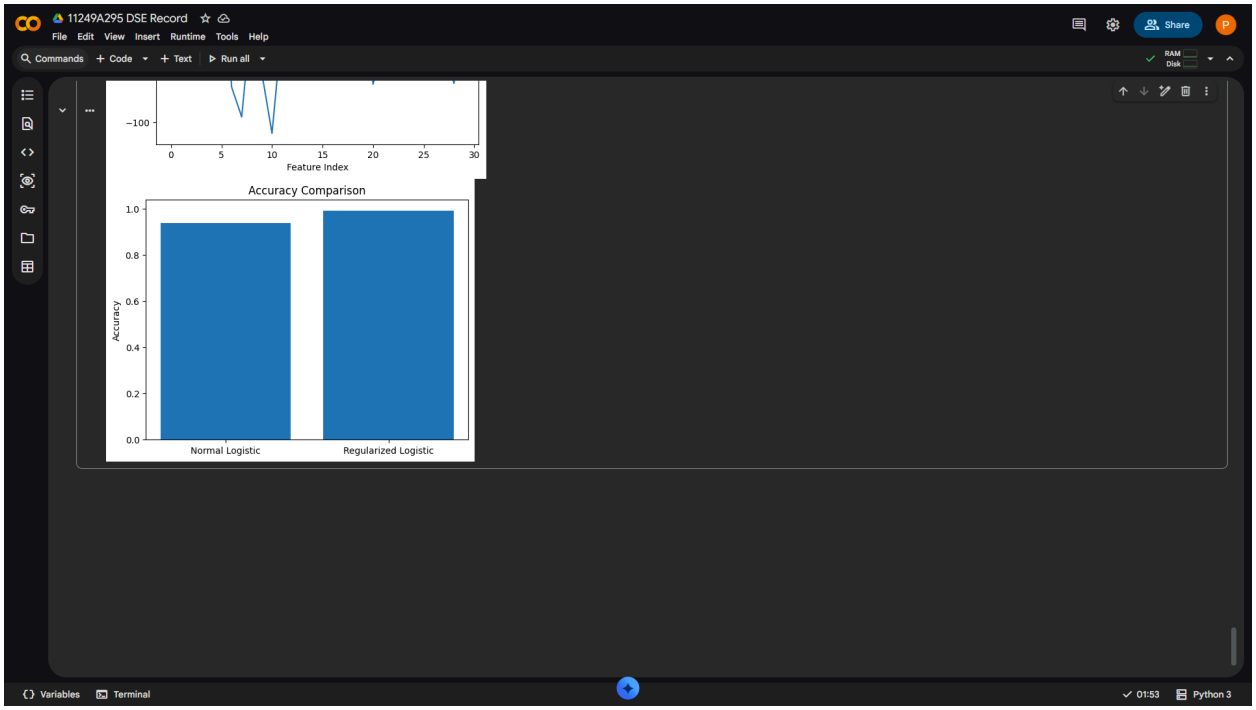
Commands + Code + Text Run all

```python
# 8. Compare Coefficients
plt.figure()
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()

# 9. Accuracy Comparison Bar Graph
plt.figure()
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]

plt.bar(models, accuracies)
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```

```
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 99.12280701754386
```



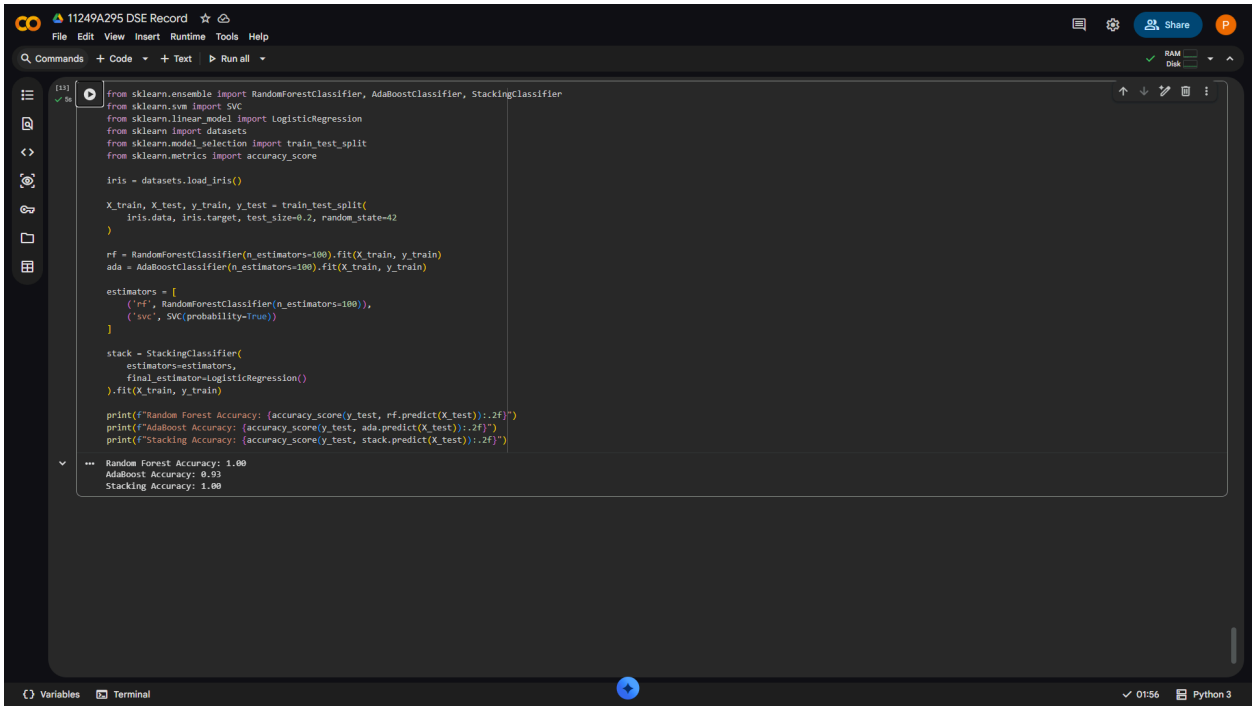
Variables Terminal 01:53 Python 3



Result : The logistic regression model was implemented successfully.

Exp 6 : Build models using different Ensembling Tecniques

Aim : To implement ensembling techniques



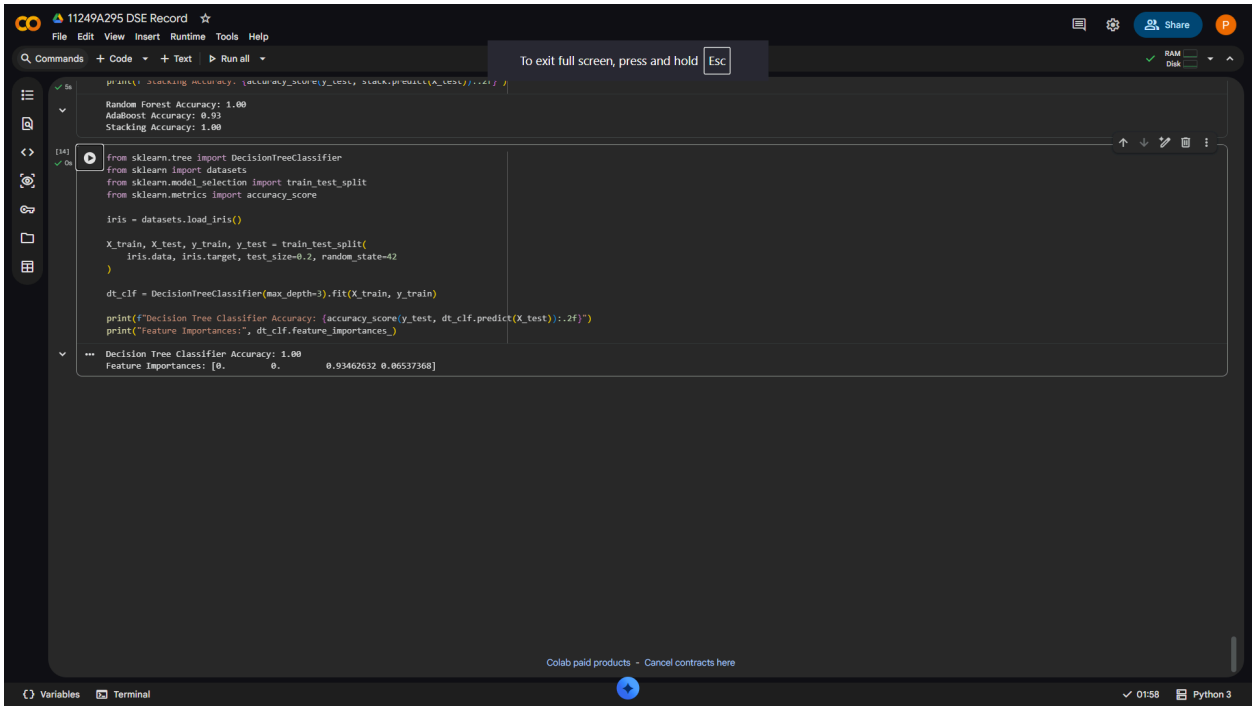
```python
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)

estimators = [
    ('rf', RandomForestClassifier(n_estimators=100)),
    ('svc', SVC(probability=True))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)

print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

Result : the ensembling techniques were implemented successfully.

Exp 7 : Build models using Decision Trees

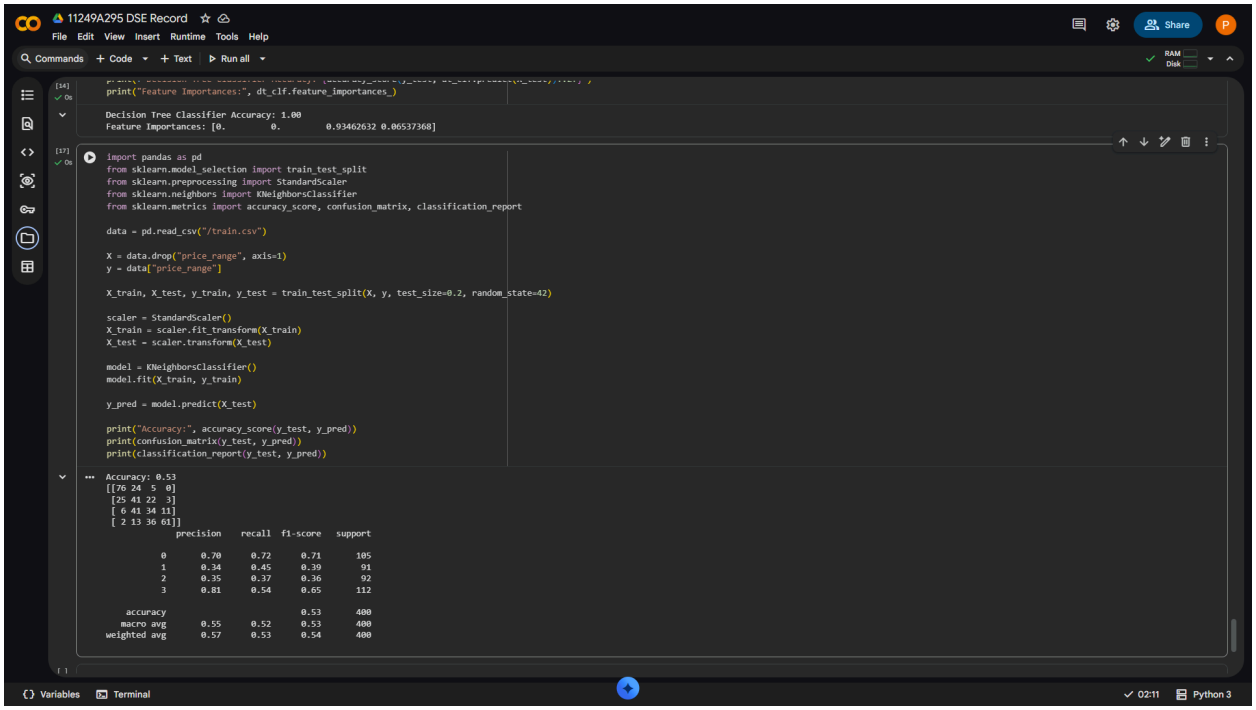
Aim : To implement decision tree models.



Result : The decision tree models were implemented successfully.

Exp 9 : Implement KNN Classification

Aim : To implement K-Nearest Neighbors algorithm



```
print("Feature Importances:", dt_clf.feature_importances_)
```

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.         0.         0.93462632 0.06537368]
```

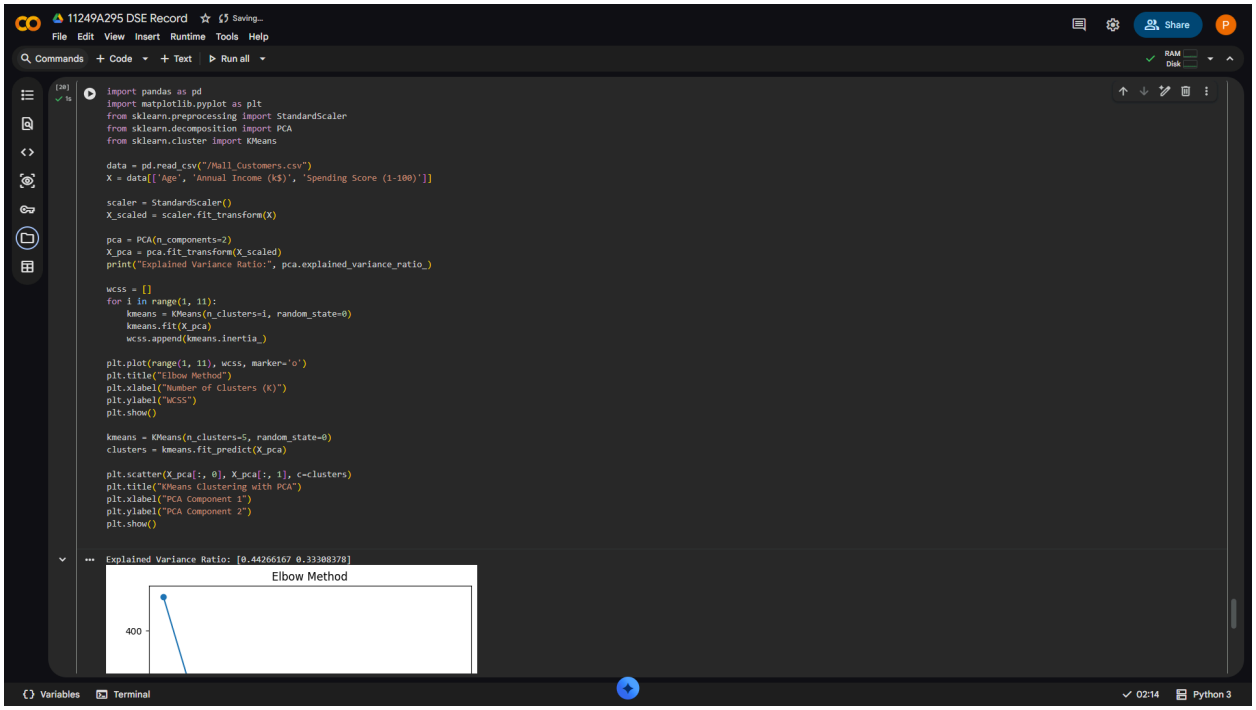
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("/train.csv")

X = data.drop("price_range", axis=1)
y = data["price_range"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

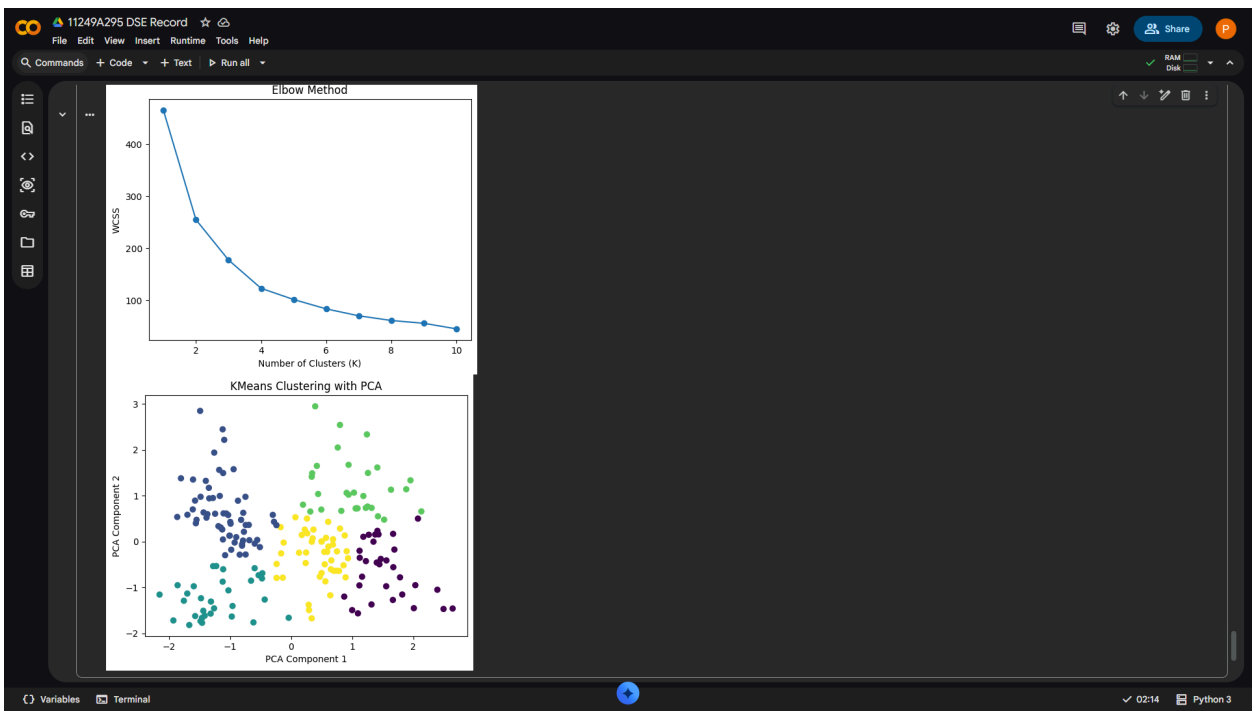
```
Accuracy: 0.53
[[76 24  5  0]
 [25 41 22  3]
 [ 6 41 34 11]
 [ 2 13 36 61]]
              precision    recall  f1-score   support

           0       0.70      0.72      0.71       105
           1       0.34      0.45      0.39        91
           2       0.35      0.37      0.36        92
           3       0.81      0.54      0.65       112

    accuracy                           0.53       400
   macro avg       0.55      0.52      0.53       400
weighted avg       0.57      0.53      0.54       400
```

Result : The KNN Classification model was implementation successfully.

Exp 10 : K-Means Clustering with PCA and Elbow Method

Aim : To implement K-Means clustering





Result : K-Means Clustering withPCA and Elbow Method was performed successfully.